



# EXAMEN: PANEL DE GESTIÓN

## "TIENDA IES PAZO"

Módulo: DWEC | IES Pazo da Mercé | Tiempo: 120 min | Puntuación: 100 pts



### FASE 0: ENTENDIENDO EL ENTORNO (LECTURA OBLIGATORIA)

Antes de programar, debes entender qué piezas tienes y qué queremos lograr. Se te entrega un archivo `index-estatico.html` que es una **maqueta**. **Restricción absoluta:** No puedes modificar la estructura del archivo HTML, salvo para añadir las etiquetas `<script>` necesarias al final del documento.

1. **El reto:** Pasar de una **maqueta estática** a una **aplicación dinámica**.

- **Estado Estático (Lo que recibes):** La página contiene datos de ejemplo "hardcodeados" en el html. Si pulsas botones o escribes en el buscador, no ocurre nada.
- **Estado Dinámico (Tu objetivo):** Tu JavaScript debe tomar el control. Debe limpiar los datos de ejemplo, traer los productos reales de la API y gestionar toda la interactividad (borrar, editar, filtrar) manipulando el DOM desde los archivos externos.
- 



## 2. ¿Cómo funciona esta página?

1. **El Contenedor Principal (#grid):** Actualmente contiene productos de ejemplo. Tu código JS debe vaciar este div y rellenarlo con datos reales de la API.
2. **El Buscador y Filtro:** Deben interceptar lo que el usuario escribe o selecciona para mostrar solo los productos que coincidan, sin que la página se recargue. También tienes un botón de buscar.
3. **El Modal Único:** Es una ventana emergente que sirve para dos cosas:
  - Si pulsas **" + Nuevo "**, el formulario debe estar vacío.
  - Si pulsas **" Editar "** en una tarjeta, el formulario debe "mágicamente" rellenarse con los datos de ese producto.
4. **Gestión de Errores (#error - container):** Es un espacio reservado en la parte superior. Si algo falla (la API no responde, por ejemplo), el usuario debe ver un mensaje de aviso allí y tú debes ver el error técnico en la consola.

## 3. El Modal y los atributos data - bs

El HTML utiliza atributos de Bootstrap llamados `data - bs` (Data-Bootstrap) para manejar la ventana emergente:

- **`data - bs - toggle = "modal"`:** Abre la ventana automáticamente al hacer clic (usado en el botón "Nuevo").
- **`data - bs - target = "#productModal"`:** Indica qué elemento debe abrirse.
- **`data - bs - dismiss = "modal"`:** Cierra el modal automáticamente (usado en botones "Cancelar" o "X").

**Tu labor:** No necesitas programar la *apertura* del modal (Bootstrap ya lo hace mediante estos atributos), pero sí debes programar el **rellenado de sus campos** cuando sea una edición y el **cierre automático** desde el controlador tras una operación exitosa.

## NOTA TÉCNICA: MANEJO DEL MODAL DESDE JAVASCRIPT

Aunque los botones del HTML ya tienen atributos para abrir el modal, tú necesitarás controlarlo desde tu código (especialmente para cerrarlo tras guardar o para abrirlo al editar).

### 1. Vincular el Modal a JavaScript

Para poder darle órdenes al modal (como "ciérrate ahora"), primero debes crear una referencia en tu archivo **View.js** usando el constructor de Bootstrap:

JavaScript

```
// Dentro del constructor de la clase View
this.modalElement = document.getElementById('productModal');
this.bsModal = new bootstrap.Modal(this.modalElement);
```

## 2. Cómo abrir y cerrar el modal por código

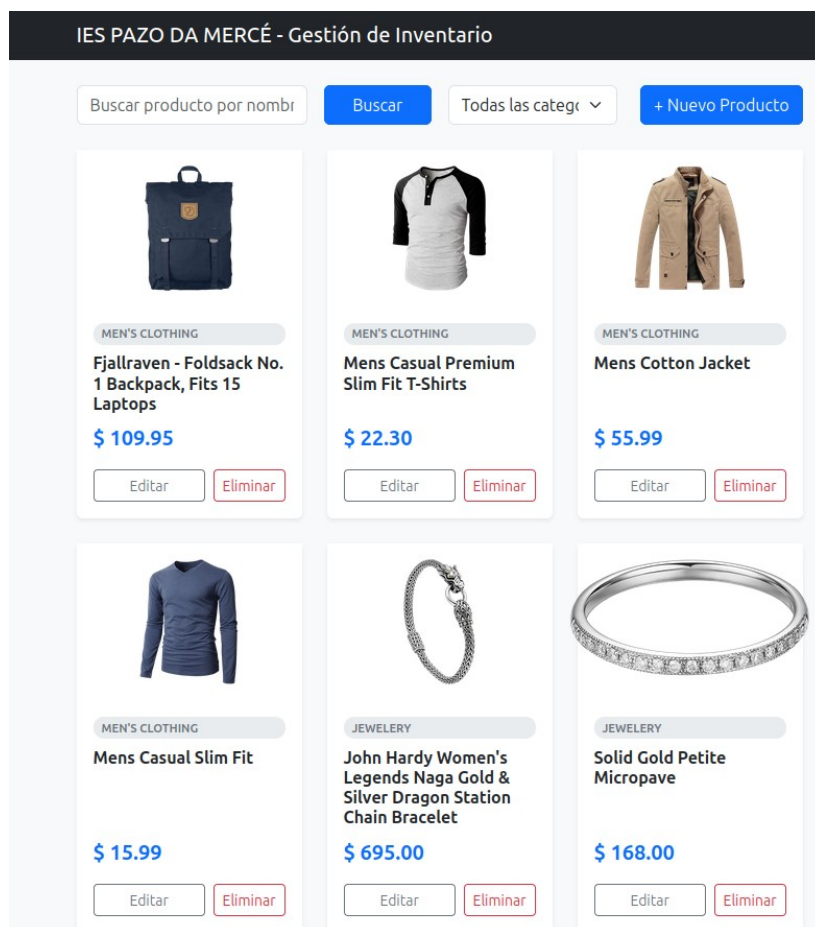
Una vez creada la referencia `this.bsModal`, puedes usar estos métodos en tus funciones de la Vista:

- `this.bsModal.show()`: Úsalo en el método de "Editar". Bootstrap abrirá la ventana y pondrá el fondo oscuro automáticamente.
- `this.bsModal.hide()`: Úsalo en el método que se ejecuta tras recibir una respuesta exitosa (ok) de la API al Guardar. Esto cerrará la ventana y limpiará el fondo.

## 3. El botón "Nuevo Producto" vs "Editar"

- **Nuevo Producto:** El botón del HTML ya tiene `data-bs-toggle="modal"`, por lo que **se abre solo**. Tu única misión es escuchar el clic para limpiar los campos del formulario.
- **Editar:** Como las tarjetas de producto se crean dinámicamente, el botón "Editar" de cada tarjeta **no abrirá el modal automáticamente**. Debes capturar el evento de la acción del botón y llamar a un método que haga el `show()`.

## 5. El producto esperado



Al agregar o editar producto:

Nuevo Producto

Nombre del Producto

Precio

Categoría

URL Imagen

Cancelar Guardar Producto

Editar Producto

Nombre del Producto

Precio

Categoría

URL Imagen

Cancelar Guardar Producto

Mensaje error en Pantalla:

IES PAZO DA MERCÉ - Gestión de Inventario

Error: Error al intentar borrar el producto en el servidor.

Buscar producto por nombr Ropa Mujer + Nuevo Producto

## DESGLOSE DETALLADO DE FASES

### FASE 1: Arquitectura MVC y Organización [15 Puntos]

- **1.1. Estructura (10 pts):** Crea los archivos correspondientes al Patron MVC separados en distintos ficheros js. Recuerda de dar el nombre apropiado a cada fichero.
- **1.2. Control de Eventos (5 pts):** Queda terminantemente prohibido el uso de onclick en el HTML. Toda la interactividad debe gestionarse desde el controlador con el manejador de eventos.

### FASE 2: Gestión de Datos y Errores [25 Puntos]

- **2.1. cargaInicial() (10 pts):** Implementa el consumo de la API (<https://fakestoreapi.com/products/>) usando comunicación asíncrona con fetch. Este método debe devolver el array de objetos recibidos.

- **2.2. Robustez y Notificación (10 pts):** Uso de try/catch. En todas tus peticiones, si la API falla o el servidor responde con error:
  - Debes imprimir el error detallado en la **consola**.
  - Debes inyectar un aviso visual (usando clases de alerta de Bootstrap) en el div #error-container del HTML.
- **2.3. Modelado (5 pts):** Crea una clase Producto para normalizar los datos (asegurar que el precio sea numérico y el ID esté presente).

### FASE 3: Visualización Dinámica (Tarjetas) [20 Puntos]

En esta fase debes implementar los métodos de manipulación del DOM:

- **3.1. mostrarProductos(productos) (5 pts):** Vaciar el #grid antes de inyectar datos para evitar duplicaciones.
- **3.2. Plantilla de Tarjeta (15 pts):** Debes usar **Template Literals** para generar el HTML. Es obligatorio que el contenedor de cada tarjeta incluya el atributo data-id="{p.id}" y los botones tengan las clases .btn-edit y .btn-delete.
  - *Se debe respetar la estructura exacta proporcionada en el examen para mantener el diseño de Bootstrap.*

JavaScript

```
/* Estructura obligatoria para mantener el diseño de Bootstrap */
<div class="col item-p" data-id="{p.id}">
  <div class="card h-100 card-product shadow-sm">
    <div class="img-box p-3 text-center">
      
    </div>
    <div class="card-body d-flex flex-column">
      <span class="category-badge mb-2 d-inline-block text-uppercase">${
{p.category}}</span>
      <h6 class="fw-bold product-name">${p.title}</h6>
      <p class="text-primary fw-bold fs-5 mt-auto">${p.price}</p>
      <div class="d-flex gap-2">
        <button class="btn btn-outline-secondary btn-sm flex-grow-1 btn-
edit">Editar</button>
        <button class="btn btn-outline-danger btn-sm btn-
delete">Eliminar</button>
      </div>
    </div>
  </div>
</div>
```

## FASE 4: Operaciones CRUD [25 Puntos]

Implementa los métodos correspondientes para la gestión de productos:

- **4.1. borrarProducto(id) (10 pts):** Al pulsar "Eliminar", capturar el ID del atributo data-id de la tarjeta, enviar petición DELETE a la API y eliminar el nodo del DOM tras confirmar el éxito.
- **4.2. Modal y Persistencia (15 pts):**
  - \* **Nuevo:** Crear el método prepararAlta() para abrir el modal con el formulario vacío. Al guardar, enviar petición POST.
  - **Editar:** Crear el método prepararEdicion(id) que cargue los datos del producto seleccionado en los inputs del modal. Al guardar, enviar petición PUT.
  - **Cierre:** El modal debe cerrarse automáticamente al finalizar la operación con éxito. (Uso obligatorio de instanciaModal.hide())

### Recuerda:

Para controlar el modal (abrir en edición o cerrar al guardar), añade esto a tu JS:

1. **Referenciarlo:** `this.bsModal = new bootstrap.Modal(document.getElementById('productModal'));`
2. **Abrirlo:** `this.bsModal.show();` (Necesario para la edición).
3. **Cerrarlo:** `this.bsModal.hide();` (Obligatorio tras guardar con éxito).

## FASE 5: Buscador y Filtros [15 Puntos]

Implementa la lógica de filtrado de información:

- **5.1. buscarProducto(texto) (7.5 pts):** Filtrado por título (evento input o botón buscar). Se debe activar mediante el evento input en el cuadro de búsqueda o al hacer clic en el botón #btnBuscar. No debe distinguir entre mayúsculas y minúsculas.
- **5.2. filtrarCategoria(categoria) (7.5 pts):** Filtrado por categoría mediante el selector #filter-cat. **Importante:** Esta operación debe realizarse sobre los datos ya cargados en memoria (array local) para optimizar el rendimiento.

## BAREMO 2ª EVALUACION

| Resultado de Aprendizaje (RA)                                                                                                | Concepto Evaluado                                                                                                            | Pts       |
|------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|-----------|
| <b>RA 7: Comunicación Asíncrona</b><br><i>Desarrollo de aplicaciones web que consumen servicios externos.</i>                | Uso de Fetch API, gestión de promesas con Async/Await y notificación de errores técnica (Consola) y de usuario (UI).         | <b>25</b> |
| <b>RA 6: Interacción con el DOM</b><br><i>Gestión dinámica de la estructura y contenido de los documentos web.</i>           | Limpieza de contenedores, inyección dinámica mediante plantillas (Template Literals) y uso de atributos data-*               | <b>20</b> |
| <b>RA 4: Arquitectura de Aplicaciones</b><br><i>Organización de código siguiendo patrones de diseño y estándares.</i>        | Implementación correcta del patrón MVC, uso de Clases ES6 y gestión centralizada de eventos (eliminación de eventos inline). | <b>25</b> |
| <b>RA 5: Gestión de Estructuras de Datos</b><br><i>Manipulación de información en el cliente mediante métodos avanzados.</i> | Uso eficiente de métodos de Array (filter, find, map) para implementar la lógica de búsqueda y filtros locales.              | <b>20</b> |
| <b>Calidad del Software</b><br><i>Mantenibilidad y buenas prácticas de programación.</i>                                     | Indentación, nombres de variables/métodos semánticos y documentación básica de funciones (JSDoc).                            | <b>10</b> |

## **FORMATO DE ENTREGA**

**Entrega un archivo .zip** con el `index.html` y la carpeta `js/` conteniendo tus archivos javascripts. Asegúrate de que el orden de carga de los scripts en el HTML sea el correcto:

**Importante:** Asegúrate de que el orden de carga de los scripts en el HTML sea el correcto para evitar errores de ejecución.